

```

package main

import (
    "os"
    "math/big"
    "strings"
    "strconv"
)

func die(msg string) {
    println(msg)
    os.Exit(1)
}

func main() {
    if len(os.Args) != 4 {
        die("Usage: ./fraction-math operand1 operator operand2")
    }

    o1str := strings.Split(os.Args[1], "/")
    if len(o1str) != 2 {
        die("Invalid operand: " + os.Args[1])
    }

    o2str := strings.Split(os.Args[3], "/")
    if len(o2str) != 2 {
        die("Invalid operand: " + os.Args[3])
    }

    operator := os.Args[2]

    a, err := strconv.Atoi(o1str[0])
    if err != nil {
        die("Invalid operand: " + os.Args[1])
    }
    b, err := strconv.Atoi(o1str[1])
    if err != nil {
        die("Invalid operand: " + os.Args[1])
    }

    o1 := big.NewRat(int64(a), int64(b))

    a, err = strconv.Atoi(o2str[0])
    if err != nil {
        die("Invalid operand: " + os.Args[1])
    }
    b, err = strconv.Atoi(o2str[1])
    if err != nil {
        die("Invalid operand: " + os.Args[1])
    }

    o2 := big.NewRat(int64(a), int64(b))

    switch operator {

```

```

case "+":
    println(o1.Add(o1, o2).RatString())
case "-":
    println(o1.Sub(o1, o2).RatString())
case "*":
    println(o1.Mul(o1, o2).RatString())
case "/":
    println(o1.Mul(o1, o2.Inv(o2)).RatString())
case ">":
    if o1.Cmp(o2) == -1 || o1.Cmp(o2) == 0 {
        println(0)
    } else if o1.Cmp(o2) == 1 {
        println(1)
    }
case "<":
    if o1.Cmp(o2) == 1 {
        println(0)
    } else if o1.Cmp(o2) == -1 || o1.Cmp(o2) == 0 {
        println(1)
    }
case "==":
    if o1.Cmp(o2) == 0 {
        println(1)
    } else {
        println(0)
    }
case "!=":
    if o1.Cmp(o2) == 0 {
        println(0)
    } else {
        println(1)
    }
case ">=":
    if o1.Cmp(o2) == 1 || o1.Cmp(o2) == 0 {
        println(1)
    } else {
        println(0)
    }
case "<=":
    if o1.Cmp(o2) == -1 || o1.Cmp(o2) == 0 {
        println(1)
    } else {
        println(0)
    }
}

```

```

}

```